# WEEK2: Calling Functions by Reference

- We learned earlier that if a variable in the main program is used as an actual argument in a function call, its value won't be changed no matter what is done to the corresponding dummy argument in the function.
- *What if we would like the function to change the main variable's contents*?

– we use pointers as dummy arguments in functions and indirect operations in the function body. (The actual arguments must then be addresses)

– Since the **actual argument variable and the corresponding dummy pointer refer to the same memory location**, changing the contents of the dummy pointer will- by necessity- change the contents of the actual argument variable.

# Calling Functions by Reference

- Call by reference with pointer arguments
  - Pass address of argument using & operator
  - Allows you to change actual location in memory
  - Arrays are not passed with & because the array name is already a pointer
- * operator
  - Used as alias/nickname for variable inside of function

```
void double( int *number )
 {
  *number = 2 * ( *number );
 }
```

  - `*number` used as nickname for the pointer passed

# Calling Functions by Reference

- The classic example of "call-by-reference" is a **swap function** designed to exchange the values of two variables in the main program. Here is a swapping program:

```
#include <stdio.h>
void swap(int *p,int *q);

main() {
   int i=3,j=9876;
   swap(&i,&j);
   printf("After swap, i=%d j=%d\n",i,j);
}

void swap(int *p,int *q) {
      int temp;
      temp=*p;
      *p=*q;
      *q=temp;
   }
```

```
After swap, i=9876 j=3
```

```c
/*Cube a variable using call-by-value */
#include <stdio.h>
int cubeByValue( int n ); /* prototype */
int main()
{
   int number = 5; /* initialize number */
   printf( "The original value of number is %d", number );
   /* pass number by value to cubeByValue */
   number = cubeByValue( number );
   printf( "\nThe new value of number is %d\n", number );
   return 0; /* indicates successful termination */
} /* end main */
/* calculate and return cube of integer argument */
int cubeByValue( int n )
{
   return n * n * n;   /* cube local variable n and return result */

} /* end function cubeByValue */
```

**OUTPUT:**

```
The original value of number is 5
The new value of number is 125
```

```c
/* Cube a variable using call-by-reference with a pointer
argument */
#include <stdio.h>
void cubeByReference( int *nPtr ); /* prototype */
int main()
{
   int number = 5; /* initialize number */
   printf( "The original value of number is %d", number );
   /* pass address of number to cubeByReference */
   cubeByReference( &number );
   printf( "\nThe new value of number is %d\n", number );
   return 0; /* indicates successful termination */

} /* end main */
/* calculate cube of *nPtr; modifies variable number in main */
void cubeByReference( int *nPtr )
{
   *nPtr = *nPtr * *nPtr * *nPtr;  /* cube *nPtr */
} /* end function cubeByReference */
```

Notice that the function prototype takes a pointer to an integer.

Notice how the address of `number` is given - `cubeByReference` expects a pointer (an address of a variable).

Inside `cubeByReference`, `*nPtr` is used (`*nPtr` is `number`).

**The original value of number is 5**
**The new value of number is 125**

Before `main` calls `cubeByValue` :

```
int main()
{
   int number = 5;

   number=cubeByValue(number);
}
```

number: 5

```
int cubeByValue( int n )
{
   return n * n * n;
}
```

n: undefined

After `cubeByValue` receives the call:

```
int main()
{
   int number = 5;

   number = cubeByValue( number );
}
```

number: 5

```
int cubeByValue( int n )
{
   return n * n * n;
}
```

n: 5

After `cubeByValue` cubes parameter `n` and before `cubeByValue` returns to `main` :

```
int main()
{
   int number = 5;

   number = cubeByValue( number );
}
```

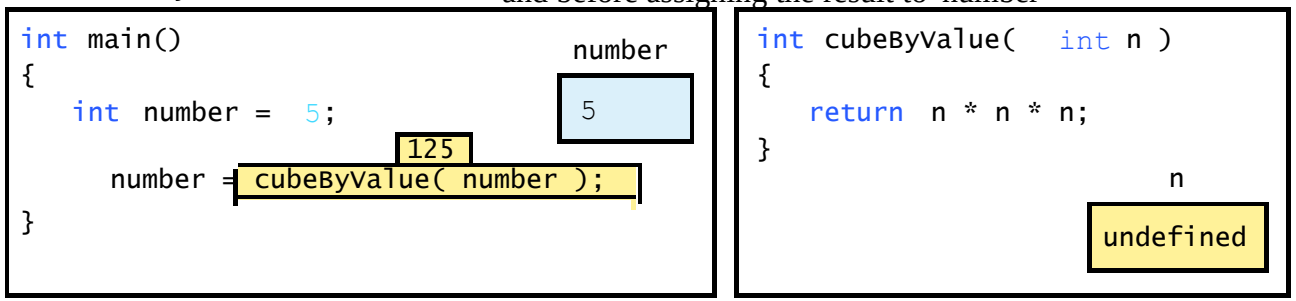
number: 5

```
int cubeByValue( int n )
{
   return n * n * n;
}
```

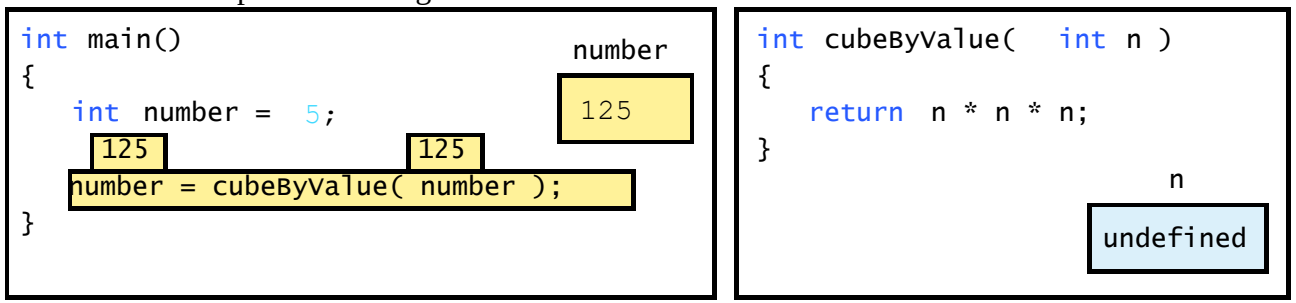
125

n: 5

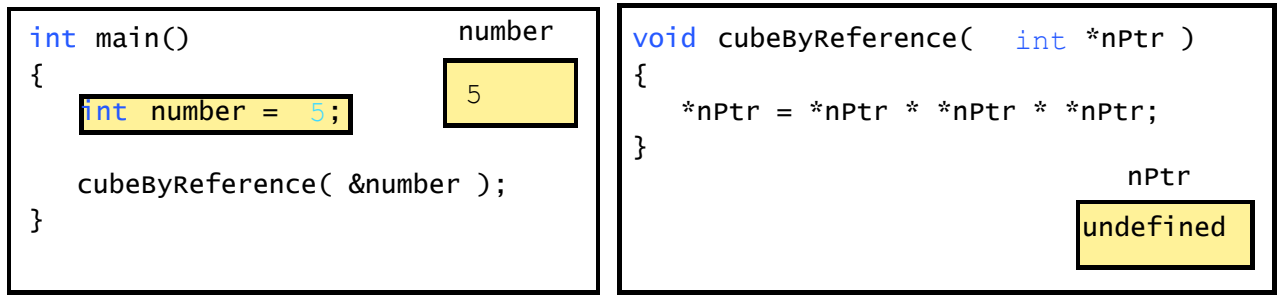
After `cubeByValue` returns to `main` and before assigning the result to `number`:



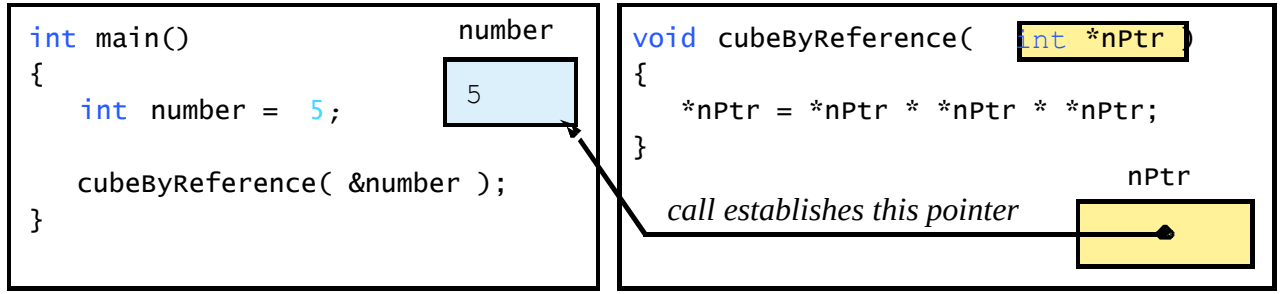

After `main` completes the assignment to `number`:



Before `main` calls `cubeByReference` :




After `cubeByReference` receives the call and before `*nPtr` is cubed:




After `*nPtr` is cubed and before program control returns to `main` :

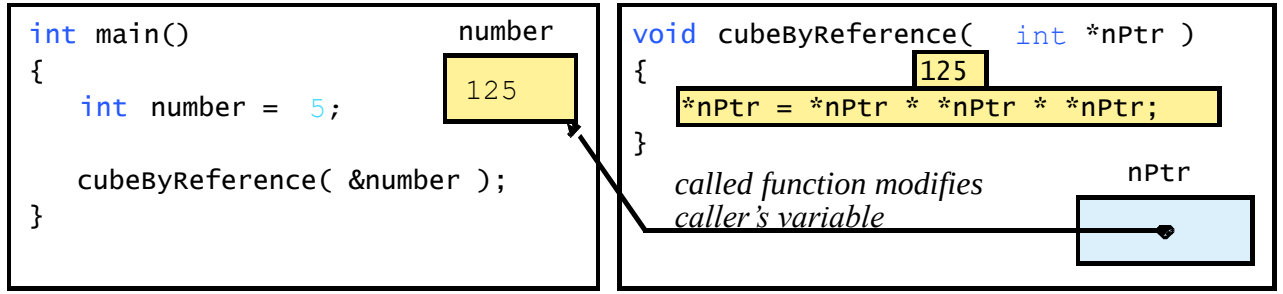
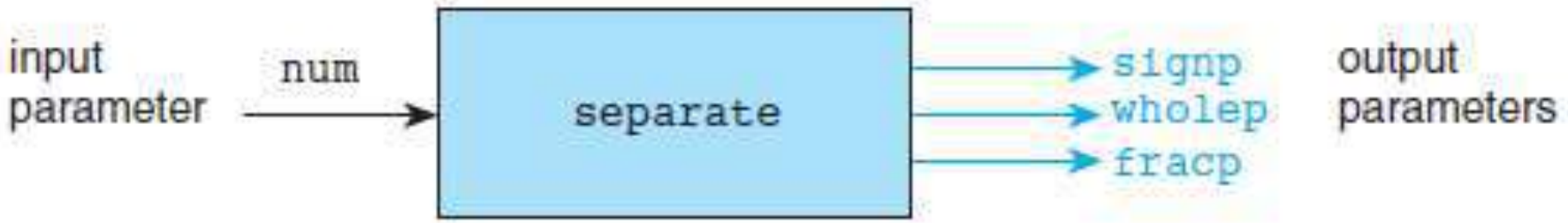



**Fig. 7.9** Analysis of a typical call-by-reference with a pointer argument.

# Figure 6.3 Function separate

```
1.  /*
2.   * Separates a number into three parts: a sign (+, -, or blank),
3.   * a whole number magnitude, and a fractional part.
4.   */
5.  void
6.  separate(double  num,    /* input - value to be split                    */
7.           char   *signp,  /* output - sign of num                         */
8.           int    *wholep, /* output - whole number magnitude of num       */
9.           double *fracp)  /* output - fractional part of num              */
10. {
11.       double magnitude; /* local variable - magnitude of num           */
12.
13.       /* Determines sign of num */
14.       if (num < 0)
15.            *signp = '-';
16.       else if (num == 0)
17.            *signp = ' ';
18.       else
19.            *signp = '+';
20.
21.       /* Finds magnitude of num (its absolute value) and
22.          separates it into whole and fractional parts                  */
23.       magnitude = fabs(num);
24.       *wholep = floor(magnitude);
25.       *fracp = magnitude - *wholep;
26. }
```

# Figure 6.4 Diagram of Function separate with Multiple Results



# Figure 6.5 Program That Calls a Function with Output Arguments

```
1. /*
2.  * Demonstrates the use of a function with input and output parameters.
3.  */
4.
5. #include <stdio.h>
6. #include <math.h>
```

*(continued)*

# Figure 6.5 Program That Calls a Function with Output Arguments (cont'd)

```
7.  void separate(double num, char *signp, int *wholep, double *fracp);
8.
9.  int
10. main(void)
11. {
12.       double value; /* input - number to analyze                          */
13.       char sn;      /* output - sign of value                            */
14.       int whl;      /* output - whole number magnitude of value          */
15.       double fr;    /* output - fractional part of value                 */
16.
17.       /* Gets data                                                        */
18.       printf("Enter a value to analyze> ");
19.       scanf("%lf", &value);
20.
21.       /* Separates data value into three parts                            */
22.       separate(value, &sn, &whl, &fr);
23.
24.       /* Prints results                                                   */
25.       printf("Parts of %.4f\n sign: %c\n", value, sn);
26.       printf(" whole number magnitude: %d\n", whl);
27.       printf(" fractional part: %.4f\n", fr);
28.
29.       return (0);
30. }
31.
32. /*
33.  * Separates a number into three parts: a sign (+, -, or blank),
34.  * a whole number magnitude, and a fractional part.
35.  * Pre: num is defined; signp, wholep, and fracp contain addresses of memory
36.  *      cells where results are to be stored
37.  * Post: function results are stored in cells pointed to by signp, wholep, and
38.  *       fracp
39.  */
40. void
41. separate(double  num,      /* input - value to be split                   */
42.          char   *signp,    /* output - sign of num                        */
43.          int    *wholep,   /* output - whole number magnitude of num      */
44.          double *fracp)    /* output - fractional part of num             */
45. {
46.       double magnitude;   /* local variable - magnitude of num           */
```

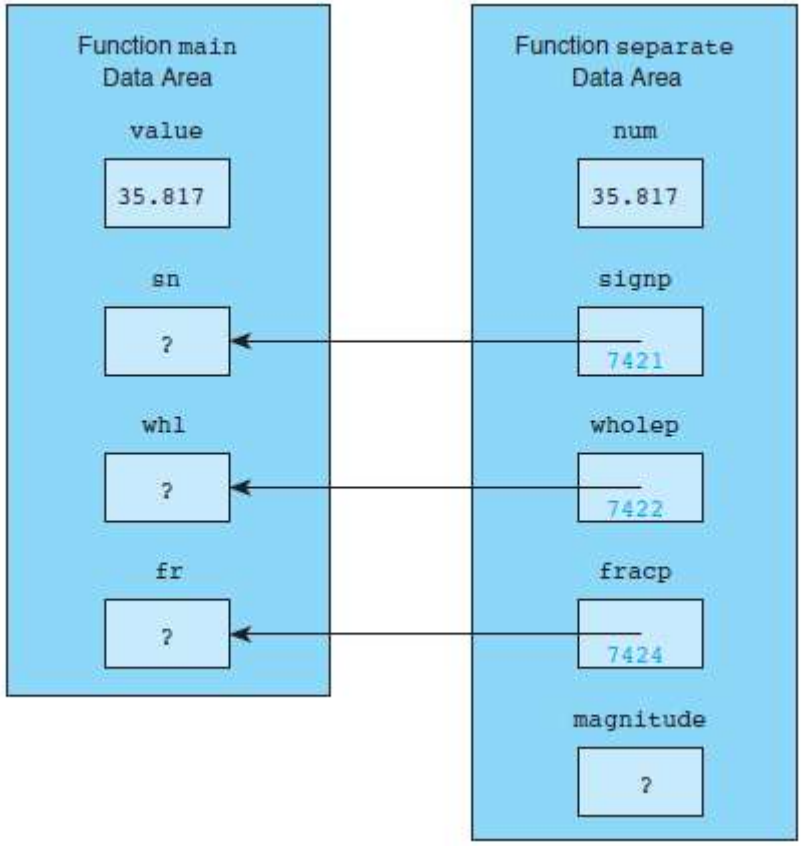
*(continued)*

# Figure 6.5 Program That Calls a Function with Output Arguments (cont'd)

```
47.       /* Determines sign of num */
48.       if (num < 0)
49.             *signp = '-';
50.       else if (num == 0)
51.             *signp = ' ';
52.       else
53.             *signp = '+';
54.
55.       /* Finds magnitude of num (its absolute value) and separates it into
56.          whole and fractional parts                                        */
57.       magnitude = fabs(num);
58.       *wholep = floor(magnitude);
59.       *fracp = magnitude - *wholep;
60. }

Enter a value to analyze> 35.817
Parts of 35.8170
  sign: +
  whole number magnitude: 35
  fractional part: 0.8170
```

# Figure 6.6 Parameter Correspondence for separate(value, &sn, &whl, &fr);



# Figure 6.7 Program to Sort Three Numbers

```
1.  /*
2.   * Tests function order by ordering three numbers
3.   */
4.  #include <stdio.h>
5.
6.  void order(double *smp, double *lgp);
7.
8.  int
9.  main(void)
10. {
11.         double num1, num2, num3; /* three numbers to put in order        */
12.
13.         /* Gets test data                                                  */
14.         printf("Enter three numbers separated by blanks> ");
15.         scanf("%lf%lf%lf", &num1, &num2, &num3);
16.
17.         /* Orders the three numbers                                        */
18.         order(&num1, &num2);
19.         order(&num1, &num3);
20.         order(&num2, &num3);
21.
22.         /* Displays results                                                */
23.         printf("The numbers in ascending order are: %.2f %.2f %.2f\n",
24.                num1, num2, num3);
25.
26.         return (0);
27. }
28.
29. /*
30.  * Arranges arguments in ascending order.
31.  * Pre:   smp and lgp are addresses of defined type double variables
32.  * Post:  variable pointed to by smp contains the smaller of the type
33.  *        double values; variable pointed to by lgp contains the larger
34.  */
35. void
36. order(double *smp, double *lgp)    /* input/output */
37. {
38.         double temp; /* temporary variable to hold one number during swap    */
```

*(continued)*

# Figure 6.7 Program to Sort Three Numbers (cont'd)

```
39.         /* Compares values pointed to by smp and lgp and switches if necessary    */
40.         if (*smp > *lgp) {
41.                 temp = *smp;
42.
43.                 *smp = *lgp;
44.                 *lgp = temp;
45.         }
46. }

Enter three numbers separated by blanks> 7.5 9.6 5.5
The numbers in ascending order are: 5.50 7.50 9.60
```

**TABLE 6.3** Trace of Program to Sort Three Numbers

| Statement | num1 | num2 | num3 | Effect |
|---|---|---|---|---|
| `scanf("...", &num1, &num2, &num3);` | 7.5 | 9.6 | 5.5 | Enters data |
| `order(&num1, &num2);` | | | | No change |
| `order(&num1, &num3);` | 5.5 | 9.6 | 7.5 | Switches `num1` and `num3` |
| `order(&num2, &num3);` | 5.5 | 7.5 | 9.6 | Switches `num2` and `num3` |
| `printf("...", num1, num2, num3);` | | | | Displays `5.5 7.5 9.6` |

Figure 6.8 Data Areas After temp = *smp;
During call order (&num1, &num3);

